ARCHIVUM

TECHNICAL REPORT

Multi-Channel RAG for Warhammer 40,000

ALESSANDRO ANGIOLILLO

Repository: `github.com/jangioj/RAG_BELLISSIMO`

1 Objective and scope

Archivum is a conversational assistant for the Warhammer 40,000 tabletop game, focused on game rules and unit information. It draws on the Core Rules booklet and a SQLite datasheet database.

The assistant can answer questions about either source, combine them when needed, and handle follow-up questions. Answers include references to the evidence used. The application runs locally and sends language-model requests to Gemini through the Google GenAI SDK.

2 Architecture

The backend is organised around `ask_rag(question, history=None, api_key=None)` in `src/rag_backend.py`. This function returns the answer together with the selected route, retrieved sources, citations, conversation history, token usage, and execution status.

A Gemini-based router first reads the question and recent conversation history. It chooses `pdf`, `sql`, or `mixed` and prepares a retrieval question for each source. The PDF channel searches the rulebook chunks, while the SQL channel generates and runs a database query. Gemini then uses the retrieved evidence to write the answer.

PDF chunks are numbered in retrieval order as `[P1]`, `[P2]`, and so on. The SQL query and its results are grouped under `[S1]`. The backend checks that the answer's source identifiers match the retrieved items.

For follow-ups, the router reads the last six messages and rewrites the question with the relevant name or context. If the conversation is about Abaddon, for example, "What weapons does he have?" becomes a question about Abaddon's weapons. Questions that combine a rule with a unit statistic use both channels. For example, determining the Wound Roll against Abaddon with a Strength 10 attack requires his Toughness from SQLite and the Wound Roll rule from the PDF.

The output uses three statuses: `ok` when the model marks the question as answerable and the response passes the output checks, `insufficient_evidence` when the system cannot support an answer, and `error` when a processing step fails.

3 Preparing the rulebook

I converted the Core Rules PDF to Markdown with Docling, then checked sampled passages against the original PDF. With support from ChatGPT-assisted comparisons, I corrected reading order, missing numbers, incomplete tables, omitted rules, and heading structure. Page-break markers were kept so that retrieved passages could be traced back to the booklet.

The corrected version is saved as `data/processed/core_rules_clean.md`. Including this file in the repository preserves the text used in the experiments, along with the manual corrections.

4 Chunking strategies

I implemented five chunking strategies to compare different ways of dividing the rulebook.

Table 1: Comparison of chunking strategies.

Strategy	Main approach	Chunks	Median words
Fixed	250-word windows with approximately 50-word overlap	126	250
Structured	Follow headings and keep paragraphs and tables together where possible	147	168
Semantic	Use similarity changes between text units, with length limits	137	195
Semantic pure	Use semantic boundaries without a fixed maximum chunk length	105	115
Hybrid	Combine semantic changes, headings, and length constraints	113	231

The semantic-pure strategy produced some very large chunks. Nineteen exceeded the embedding model's 512-token limit, with a maximum of 2,382 tokens. I excluded this strategy from the main retrieval comparison to avoid truncating the text during embedding.

I compared the remaining four strategies on the same development questions. Structured + RRF achieved the highest full anchor coverage, at 0.949, with anchor recall of 0.971 and a median retrieved context of 906 words. I therefore selected `structured_chunks.jsonl` for the application.

5 PDF retrieval

I compared BM25, dense retrieval, and Reciprocal Rank Fusion (RRF).

BM25 ranks chunks through word matches, taking term frequency and chunk length into account. Dense retrieval uses `BAAI/bge-small-en-v1.5` to encode questions and chunks, then ranks the normalized embeddings by cosine similarity. Document embeddings are stored locally and reused between queries.

RRF combines the two rankings using each chunk's position. The application retrieves 20 candidates from each retriever, applies RRF with a rank constant of 60, and sends the top five structured chunks to the answer generator.

I also tested adding neighboring chunks. This raised full anchor coverage on the development set from 0.949 to 0.974 and increased the median context from 906 to 1,236 words. The final backend uses the five RRF results without expansion, keeping the context smaller.

6 SQLite retrieval

For database questions, Gemini receives a retrieval question and a compact description of the schema. The prompt explains relevant details, such as statistics stored as text and joins between related tables. It returns a SQL query with named parameters, which is then passed to a separate executor.

The executor opens SQLite in read-only mode, restricts access to allowed tables and functions, and limits query execution and the number of returned rows.

A query can return up to 100 rows. The answer generator receives up to the first 20, together with the query, column names, and an indication of whether the result is incomplete. The backend keeps the full returned result in its source record.

7 Conversation, interface, and diagnostics

The browser interface is served from `dist/` by `http_adapter.py`, which connects it to the Python backend. It opens at `http://127.0.0.1:8000` by default. Users enter a Gemini API key and receive answers with source references and short source summaries. The key and conversation history are held in server memory for the session.

The backend records token usage for routing, SQL generation, and answer generation when reported by Gemini. Evaluation scripts also save elapsed time, retrieved evidence, and diagnostic fields to help inspect failures and resource use.

8 Tests and evaluation design

I evaluated Archivum in two stages, first testing PDF retrieval and then the complete chatbot.

The PDF retrieval benchmark contained 51 questions: 39 development questions and 12 held-out questions. I used the development set to compare four chunking strategies and three retrieval methods, each returning five chunks. The selected structured + RRF configuration was then evaluated on the held-out set. Reference evidence was marked with page numbers and short text anchors, allowing the same questions to be used across chunking strategies.

The retrieval metrics were Anchor Hit@5, Anchor Recall@5, Full Anchor Coverage@5, and Anchor MRR@5. Together, they measure how much of the required evidence was retrieved and how early it appeared in the ranking.

The chatbot benchmark contained 30 manually checked questions, split into 17 development and 13 held-out cases. These covered general rules, database facts, questions requiring both sources, follow-ups, and questions outside the available material. I used the development cases to improve the system and froze the backend before running the held-out evaluation.

I recorded routing decisions, execution status, and retrieved evidence for each case. RAGAS then scored completed answers for faithfulness to the evidence and factual correctness F1 against a reference answer. These averages include non-empty answers with `status=ok`; abstentions and errors are reported separately.

9 Results

9.1 PDF retrieval

Structured + RRF gave the following results:

Table 2: PDF retrieval results for structured + RRF.

Split	Questions	Anchor Hit@5	Anchor Recall@5	Full Anchor Coverage@5
Development	39	1.000	0.971	0.949
Held-out	12	0.917	0.917	0.917

This configuration retrieved all required anchors for 11 of the 12 held-out questions. It also kept the retrieved context relatively short, with a median of 906 words on the development set.

9.2 Complete chatbot

Table 3: End-to-end chatbot results.

Split	Cases	Correct routes	Answers with status=ok	Faithfulness	Factual correctness F1
Development	17	17/17	13/17	0.9231	0.9054
Held-out	13	11/11	10/13	0.9475	0.7040

The router selected the expected sources for every question with a reference route. Two held-out questions had no reference route, which explains the denominator of 11. The RAGAS averages cover 13 development answers and ten held-out answers.

The held-out run produced ten answers with `status=ok`, two insufficient-evidence outcomes, and one router error. Faithfulness remained high, while factual correctness was lower than on the development set. On manual review, I judged nine answers substantially correct and one abstention appropriate.

10 Error analysis and limitations

Several errors came from incomplete retrieval. Some questions required rules from different parts of the booklet, and the retrieved chunks occasionally missed a necessary condition. Unit names also caused problems in SQLite. For example, a search for "Ork Boyz" returned no results because the datasheet was named "Boyz".

There were also errors in the generated reasoning. One answer first gave a Wound Roll of 2+ and then corrected it to 3+, leaving a contradiction in the final response. Questions outside the available sources were handled unevenly: some led to a clear abstention, while another caused a router error.

The benchmark was small, and RAGAS used the same Gemini model as the chatbot, `gemini-3.5-flash-lite`. Manual review was useful here, particularly for checking answers whose automated scores seemed too generous.

The project was also shaped by the practical constraints of an academic assignment. The available Gemini API quota limited the number of experiments and evaluation runs I could carry out. I therefore had to simplify the workflow and accept unresolved issues that I would otherwise have addressed before submission. Further experimentation would have required additional API calls and evaluation runs. Paid access could have reduced the delays caused by quota limits, but it was beyond my budget. Delivering a working project on time therefore took priority over further experimentation and refinement.

11 Conclusion

Archivum brings together rulebook and database information in a conversational assistant. It can combine the two sources, interpret follow-up questions, and show the evidence used in its answers.

The results were strongest in routing and PDF evidence retrieval. The remaining work centres on retrieving complete rules, matching unit names more reliably, and improving the consistency of generated answers.